

CS341 CacheLab Design

Tomas Salaz

5 November 2025

Data Structures

For implementing a Cache for the simulation, I will use 3 structs to represent the Cache, **Line**, **Set**, **Cache**. These are all we need since we are not concerned with the Block Offset according to the Assignment document.

The **Line** struct will have the following variables: `int valid; unsigned long tag; unsigned long lru;`

These variables are self explanatory for this structure.

The **Set** struct is only going to have an array of lines, so `Line *Lines;` is going to be the only necessary variable.

The **Cache** struct is going to be the main container for the other structs and will have the following fields: `int s,S,E,b; Set *Sets; unsigned long his, misses, evicts, tick;`

The Ints are used to pass in the arguments defined in the assignment PDF. The Array of Sets contains the Lines and imitates the Cache structure. The rest of the fields are used for measuring and LRU implementation.

Algorithms

2.1 Address Decode

Given a 64-Bit address with the params **s,E,b**, we need to calculate the set index and the tag. We can do this with bit shifting and masking and would look like something below. This is like the partitioning we do in the handwritten exercises in class.

$$\text{setIdx} = (\text{address} \gg b) \ ((1 \ll s) - 1)$$

$$\text{tag} = \text{address} \ll (s + b)$$

2.2 Access Routine

For now this is going to be conceptualized in 1 method, but implementation may need to have more modularity. Since this is a complex operation I am going to

outline the flow.

1. Parse Argv to get `setIdx`, `tag`.
2. Initialize the set array for the new data in the current cache and update the LRU.
3. Check for Cache Hit and update the hit counter if needed, would be in a loop, trying to use spatial locality for quick checking.
4. Check for miss in the same manner, using Loop and Spatial Locality.
5. Check Current cache if Miss and update data using set from step 2.

2.3 Trace Process

To simulate the valgrind trace file we have to implement code to imitate the output. We need to first get the options with the flags. Once we have that we need to allocate space for the cache, would use `calloc` since cache is stored in arrays.

Once we get the args and initialize the cache we would have to read lines, with in order and use a method `printSummary()`, to print the hits, misses, and evictions based on the I, L, S, M, and -v inputs.